**Breadth First Search (BFS)**

**Aim**

To write a C program to implement **Breadth First Search (BFS)** traversal of a graph using an **adjacency matrix and queue**.

**Algorithm**

1. **Start** the program.
2. Read the number of vertices n.
3. Read the **adjacency matrix** of the graph.
4. Initialize all vertices as **not visited** by setting visited[i] = 0.
5. Read the **starting vertex**.
6. Mark the starting vertex as visited.
7. Insert the starting vertex into the queue.
8. Repeat the following steps while the queue is not empty:
    - Remove a vertex from the front of the queue.
    - Display the removed vertex.
    - Check all vertices connected to it.
    - If an adjacent vertex is not visited:
        - Mark it as visited.
        - Insert it into the queue.
9. Continue until the queue becomes empty.
10. **Stop** the program.

**Program**

```
#include <stdio.h>

#define MAX 50

int main()
{
   int graph[MAX][MAX];
   int visited[MAX];
   int queue[MAX];
```

```
int n, i, j;
int front = 0, rear = 0;
int start, current;

// Read number of vertices
printf("Enter number of vertices: ");
scanf("%d", &n);

// Read adjacency matrix
printf("Enter the adjacency matrix:\n");

for (i = 0; i < n; i++)
{
   for (j = 0; j < n; j++)
   {
      scanf("%d", &graph[i][j]);
   }
}

// Initialize visited array
for (i = 0; i < n; i++)
{
   visited[i] = 0;
}

// Read starting vertex
printf("Enter starting vertex: ");
scanf("%d", &start);

// Insert starting vertex into queue
visited[start] = 1;
queue[rear] = start;
rear++;

printf("BFS Traversal: ");

// BFS traversal
while (front < rear)
{
   // Remove vertex from queue
   current = queue[front];
   front++;

   printf("%d ", current);

   // Find adjacent vertices
   for (i = 0; i < n; i++)
```

```
    {
      if (graph[current][i] == 1 && visited[i] == 0)
      {
        visited[i] = 1;
        queue[rear] = i;
        rear++;
      }
    }
  }

  return 0;
}
```

**Sample Input**

Enter number of vertices: 5

Enter the adjacency matrix:

0 1 1 1 0

1 0 1 0 0

1 1 0 0 1

1 0 0 0 0

0 0 1 0 0

Enter starting vertex: 0

**Output**

BFS Traversal: 0 1 2 3 4

**Result**

Thus, the **Breadth First Search (BFS) traversal of the graph was successfully implemented using an adjacency matrix and queue**, and the vertices were visited level by level.

## Depth First Search (DFS)

**Aim**

To write a C program to implement **Depth First Search (DFS)** traversal of a graph using an **adjacency matrix and recursion**.

**Algorithm**

1. **Start** the program.
2. Read the number of vertices n.
3. Read the **adjacency matrix** of the graph.
4. Initialize all vertices as **not visited** by setting visited[i] = 0.
5. Read the **starting vertex**.
6. Call the DFS function with the starting vertex.
7. In the DFS function:
    - Mark the current vertex as visited.
    - Display the current vertex.
    - Check all vertices connected to the current vertex.
    - If an adjacent vertex is not visited, call DFS for that vertex.
8. Continue until all reachable vertices are visited.
9. **Stop** the program.

**Program**

```
#include <stdio.h>
#define MAX 50
int graph[MAX][MAX];
int visited[MAX];
int n;
void DFS(int vertex)
{
   int i;

   // Mark vertex as visited
```

```
    visited[vertex] = 1;
    // Display vertex
    printf("%d ", vertex);

    // Visit adjacent vertices
    for (i = 0; i < n; i++)
    {
        if (graph[vertex][i] == 1 && visited[i] == 0)
        {
            DFS(i);
        }
    }
}
int main()
{
    int i, j;
    int start;
    // Read number of vertices
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    // Read adjacency matrix
    printf("Enter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    // Initialize visited array
    for (i = 0; i < n; i++)
    {
        visited[i] = 0;
    }
    // Read starting vertex
    printf("Enter starting vertex: ");
    scanf("%d", &start);
    printf("DFS Traversal: ");
    DFS(start);
    return 0;
}
```

**Sample Input**

Using the same graph as the BFS example:

Enter number of vertices: 5

Enter the adjacency matrix:

0 1 1 1 0

1 0 1 0 0

1 1 0 0 1

1 0 0 0 0

0 0 1 0 0

Enter starting vertex: 0

**Output**

DFS Traversal: 0 1 2 4 3

**Note:** DFS output can change depending on the order in which adjacent vertices are checked. In this program, vertices are checked from 0 to n-1.

**Result**

Thus, the **Depth First Search (DFS) traversal of the graph was successfully implemented using an adjacency matrix and recursion**, and the vertices were visited by going as deep as possible before backtracking.